

数据库系统 Ch7 —ER 实体关系模型

详细学习笔记

基于：数据库系统 PPT Ch7 Entity-Relation Model (116 页)

2026 年 6 月 21 日

目录

1 数据库设计流程 (Database Design Process) [p.3–4]	2
1.1 主要考点	2
2 ER 基本概念 (Basic ER Concepts) [p.5–14]	2
2.1 核心定义	2
2.2 属性类型 [p.13]	2
3 约束 (Constraints) [p.15–33]	3
3.1 映射基数 (Mapping Cardinality) [p.15–17]	3
3.2 码 (Keys) [p.18–19]	3
3.3 参与约束 (Participation Constraints) [p.29–30]	3
3.4 三元联系约束 [p.32]	3
4 ER 图符号体系 (ER Diagram Notation) [p.20–31]	4
4.1 基数约束在 ER 图中的表示 [p.25–28]	4
5 设计问题 (Design Issues) [p.34–38]	5
5.1 实体集 vs 属性 [p.34, 59–60]	5
5.2 实体集 vs 联系集 [p.34]	5
5.3 二元 vs n 元联系 [p.34–37]	5
5.4 联系属性放置 [p.38]	5
6 弱实体集 (Weak Entity Sets) [p.44–48]	5
6.1 定义 [p.44]	5

6.2	ER 图表示 [p.45–46]	6
6.3	实例	6
6.4	强实体 vs 弱实体对比 [p.48]	6
7	扩展 ER 特性 (Extended E-R Features) [p.49–57]	6
7.1	特殊化 (Specialization) [p.49]	6
7.2	概化 (Generalization) [p.51]	6
7.3	ISA 约束 [p.52–54]	7
7.3.1	成员约束 (Membership Constraint) [p.53]	7
7.3.2	不相交性 (Disjointness) [p.53]	7
7.3.3	完备性约束 (Completeness Constraint) [p.54]	7
8	ER 设计方法论 (ER Design Methodology) [p.58–80]	7
8.1	三条设计原则 [p.59]	7
8.2	设计步骤 [p.64]	7
8.2.1	局部 ER 设计 6 步 [p.66–68]	7
8.2.2	全局集成：三类冲突 [p.71–72]	8
8.2.3	全局集成方法 [p.74]	8
8.2.4	优化 [p.76–77]	8
8.3	完整实例 [p.65–77]	8
8.4	学校练习 [p.78–80]	8
8.5	在线学习平台完整设计实例 [p.41–43]	9
9	UML 类图 (UML Class Diagrams) [p.81–85]	9
9.1	UML vs ER 图 [p.81–85]	9
9.2	UML 基数约束 [p.85]	9
10	ER 到关系模式的转换 (Reduction to Relation Schemas) [p.86–99]	9
10.1	数据库设计三阶段 [p.87]	9
10.2	6 条核心转换原则 [p.96]	9
10.3	具体规则详解	10
10.4	特殊化/概化转换 [p.94–95]	10
10.5	综合练习 [p.97–99]	10
11	弱实体复习要点 [p.106]	11

1 数据库设计流程 (Database Design Process) [p.3–4]

1.1 主要考点

- 数据库设计四阶段 [p.3]:
 1. 需求分析 (Requirement Analysis): DB 设计师与领域专家和用户深入交互 → 产出用户需求规格说明
 2. 概念设计 (Conceptual Design): 选择数据模型, 将需求转化为概念模式 → 产出 E-R 图
 3. 逻辑设计 (Logical Design): 将概念模式转化为数据库的模式 (如关系模式 Relation Schema)
 4. 物理设计 (Physical Design): 选择 DBMS, 确定文件组织形式和内部存储结构

2 ER 基本概念 (Basic ER Concepts) [p.5–14]

2.1 核心定义

- 实体 (Entity): 客观存在且可相互区分的事物, 具有属性。例: 特定的人、公司、事件 [p.5]
- 实体集 (Entity Set): 具有相同类型、共享相同属性的实体集合。例: 所有 person、company [p.5]
- 属性 (Attribute): 实体集中所有成员具有的描述性特性。例: people 有 names 和 addresses [p.5]
- 域 (Domain): 每个属性允许的取值集合 [p.13]
- 联系 (Relationship): 多个实体之间的关联 [p.7]
- 联系集 (Relationship Set): 相同类型联系的集合, 是 n 个实体集上的数学关系:

$$\{(e_1, e_2, \dots, e_n) \mid e_1 \in E_1, e_2 \in E_2, \dots, e_n \in E_n\}$$

[p.7]

- 联系的度 (Degree): 参与联系的实体集个数。二元 (Binary) 最常见 (度 = 2), 三元 (Ternary) 很少见 (度 = 3) [p.11]
- 角色 (Role): 实体在联系中所起的作用。当相同实体集以不同身份多次参与同一联系时, 必须用角色指明语义。例: employee 通过 works_for 联系关联自身时, manager 和 worker 是两个角色 [p.12]
- 联系集本身也可以有属性。例: depositor 联系集有 access-date 属性 [p.10]

2.2 属性类型 [p.13]

- 简单属性 (Simple Attribute): 不可再分的原子属性
- 复合属性 (Composite Attribute): 可分解为更小部分, 如 name → first_name + last_name
- 单值属性 (Single-valued Attribute): 每个实体只有一个值
- 多值属性 (Multi-valued Attribute): 每个实体可有多个值, 如 phone_numbers。ER 图中用双椭圆表示

- **派生属性 (Derived Attribute)**: 可从其他属性计算得出, 如 age 可由 date_of_birth 计算。ER 图中用虚线椭圆表示

3 约束 (Constraints) [p.15–33]

3.1 映射基数 (Mapping Cardinality) [p.15–17]

描述二元联系集中实体间的数量对应关系, 必须是以下四种之一:

- **一对一 (One-to-one, 1:1)**: A 中每个实体至多与 B 中一个实体关联, 反之亦然
- **一对多 (One-to-many, 1:M)**: A 中每个实体可与 B 中多个关联, B 中每个实体至多与 A 中一个关联
- **多对一 (Many-to-one, M:1)**: A 中每个实体至多与 B 中一个关联, B 中每个实体可与 A 中多个关联
- **多对多 (Many-to-many, M:N)**: 双方均可关联多个

3.2 码 (Keys) [p.18–19]

- **超码 (Super Key)**: 能唯一确定每个实体的一个或多个属性的集合
- **候选码 (Candidate Key)**: 最小的超码 (Minimal Super Key)。例: customer_id 是 customer 的候选码
- **主码 (Primary Key)**: 从多个候选码中选出的一个
- **联系集的码**: 参与实体集主码的组合构成联系集的超码。这意味着在同一联系集中, 一对实体最多只能有一个联系实例 [p.19]
- 联系集候选码的选择需考虑联系集的语义和映射基数 [p.19]

3.3 参与约束 (Participation Constraints) [p.29–30]

- **全部参与 (Total Participation)**: 实体集中每个实体都至少参与一个联系。ER 图中用双线表示。例: loan 在 borrower 中是全部参与 (每笔贷款必须有客户) [p.29]
- **部分参与 (Partial Participation)**: 部分实体可以不参与任何联系。ER 图中用单线表示。例: customer 在 borrower 中是部分参与 [p.29]
- **基数限制也可表示参与约束 [p.30]**: 通过在 ER 图上标注基数范围 (min..max) 可同时表达基数约束和参与约束

3.4 三元联系约束 [p.32]

- 三元 (或更高阶) 联系中最多允许一个箭头
- 超过一个箭头会产生语义歧义, 因此明确规定禁止多于一个箭头
- **两种歧义解释** (若 R 在 A、B、C 间有指向 B 和 C 的两个箭头):
 1. 每个 A 实体关联唯一的 (B, C) 组合

- 2. 每对 (A, B) 关联唯一的 C, 且每对 (A, C) 关联唯一的 B
- 不同形式化方法曾采用不同解释, 为避免混淆, 统一禁止多箭头
- **易错点:** 三元联系中不能用箭头方向直接推断 1:1:M 这样的复合基数约束

4 ER 图符号体系 (ER Diagram Notation) [p.20–31]

[需要图示: 参见源文件 p.20–21 ER 图符号总览、p.25–28 四种基数约束完整图、p.29 全部/部分参与对比图]

元素	符号	说明
实体集 (Entity Set)	矩形 (Rectangle)	
弱实体集	双矩形 (Double Rectangle)	[p.45]
联系集 (Relationship Set)	菱形 (Diamond)	
标识联系	双菱形 (Double Diamond)	[p.45]
属性 (Attribute)	椭圆 (Ellipse)	
多值属性	双椭圆 (Double Ellipse)	[p.21]
派生属性	虚线椭圆 (Dashed Ellipse)	[p.21]
主码	下划线 (Underline)	
分辨符	虚线下划线 (Dashed Underline)	[p.45]
基数”一”	有向线 ($\rightarrow$)	
基数”多”	无向线 ($-$)	
全部参与	双线 (Double Line)	[p.29]
角色	连线上的标签	[p.24]

4.1 基数约束在 ER 图中的表示 [p.25–28]

- 1:1 联系: 两边均有箭头
- 1:M 联系: 仅”一”侧有箭头, ”多”侧无箭头
- M:1 联系: 仅”一”侧有箭头
- M:N 联系: 两边均无箭头
- **易错点:** 箭头总是指向”一”侧。1:M 中箭头指向 1 侧; M:1 中箭头也指向 1 侧 (即 M 侧实体到 1 侧实体是 M:1)。混淆方向是常见错误
- **解题技巧:** 判断基数时先分别问”一个 A 对应几个 B”和”一个 B 对应几个 A”, 再组合得出类型

5 设计问题 (Design Issues) [p.34–38]

5.1 实体集 vs 属性 [p.34, 59–60]

属性升级为实体集的两个判断条件（满足其一即可）：

1. 属性不再具有需要描述的性质（属性在含义上是不可分的数据项）
2. 属性不能再与其他实体集具有联系
 - 例：工资最初作为职工属性；当需要描述基本工资/补贴/各种扣除时 → 变为独立实体集 [p.61]
 - 例：仓库 → 若只与货物有存放关系可作为货物属性；若与职工发生保管联系 → 变为独立实体集 [p.62–63]

5.2 实体集 vs 联系集 [p.34]

联系集应用于描述实体间发生的动作 (action)。

5.3 二元 vs n 元联系 [p.34–37]

- n 元联系能更清晰地展示多个实体参与单一联系
- 非二元转二元通用方法 [p.36]：创建人工实体集 E ，拆为三个二元联系 $R_A(E, A)$ 、 $R_B(E, B)$ 、 $R_C(E, C)$ 。原联系属性归入 E
- 约束的翻译可能不完整，可通过让 E 成为由三个联系集标识的弱实体集来避免创建人造标识属性 [p.37]
- 二元拆分的优势 [p.35]：可表达部分信息。例：三元联系 `parents(child, father, mother)` 拆为 `father` 和 `mother` 两个二元联系后，可以只记录已知的母亲而不必同时知道父亲
- 何时保留 n 元 [p.35]：有些联系天然是非二元的，如 `works_on(employee, job, branch)`，适合保留三元形式

5.4 联系属性放置 [p.38]

若联系从 `account` 到 `customer` 是 many-to-one，则 `access-date` 可放入 `account` 侧。

6 弱实体集 (Weak Entity Sets) [p.44–48]

6.1 定义 [p.44]

- 弱实体集 (Weak Entity Set)：没有主键 (Primary Key) 的实体集
- 存在依赖于标识实体集 (Identifying/Strong Entity Set)
- 必须通过全部参与 + 一对多的标识联系关联到强实体
- 分辨符/部分键 (Discriminator/Partial Key)：弱实体集内部区分不同实体的属性集
- 弱实体集主码 = 强实体主码 + 分辨符

6.2 ER 图表示 [p.45–46]

- 弱实体集：双矩形
- 标识联系：双菱形
- 分辨符：虚线下划线
- 强实体主码不显式存储在弱实体中：标识联系已隐含该关系，显式存储会与联系冗余 [p.46]

6.3 实例

- **payment 弱实体** [p.45]：强实体 loan，分辨符 payment_number，主码 (loan_number, payment_number)
- **course_offering 弱实体** [p.47]：强实体 course，分辨符 semester + section_number

[需要图示：参见源文件 p.45 payment 弱实体 ER 图（双矩形 + 双菱形 + 虚线下划线）]

易错点：弱实体的主码构造中，强实体主码部分来自标识联系而非显式属性列——不要将 loan_number 作为 payment 的属性列重复定义 [p.46]

6.4 强实体 vs 弱实体对比 [p.48]

特性	强实体集	弱实体集
存在性	可以独立存在	依赖于强实体集
主键	拥有自己的唯一主键	强实体主键 + 分辨符
标识方法	由自己的主键唯一标识	由强实体主键 + 分辨符共同标识
ER 图表示	矩形	双矩形
联系表示	菱形	双菱形（标识联系）

何时不用弱实体集：如果实体拥有全局唯一 ID（如身份证号、UUID），应建模为强实体 [p.48]

7 扩展 ER 特性 (Extended E-R Features) [p.49–57]

7.1 特殊化 (Specialization) [p.49]

- 自顶向下：在实体集内标识子分组，形成低层实体集
- ISA 三角标注，也称超类-子类关系 (Superclass-Subclass Relationship) [p.52]
- 属性继承 (Attribute Inheritance)：低层实体集自动继承高层实体集的所有属性和联系参与

7.2 概化 (Generalization) [p.51]

- 自底向上：合并共享相同特性的实体集为高层实体集
- 特殊化与概化互为逆操作，ER 图中表示相同，术语可互换

7.3 ISA 约束 [p.52–54]

- **多重特殊化 (Multiple Specializations)**: 同一实体集可基于不同特征有多套分类体系。例: employee 可按雇佣类型 (permanent/temporary) 和岗位角色 (officer/secretary/teller) 两套分类 [p.52]

7.3.1 成员约束 (Membership Constraint) [p.53]

- **条件定义 (Condition-defined)**: 基于条件自动判定。例: $\text{age} > 65 \rightarrow \text{senior-citizen}$
- **用户定义 (User-defined)**: 人工指定

7.3.2 不相交性 (Disjointness) [p.53]

- **不相交 (Disjoint)**: 一个实体只能属于一个低层实体集。标注 "disjoint"
- **有重叠 (Overlapping)**: 一个实体可属于多个低层实体集

7.3.3 完备性约束 (Completeness Constraint) [p.54]

- **total**: 高层实体必须属于至少一个低层实体集
- **partial**: 高层实体可以不属任何低层实体集

Disjoint/Overlapping 与 Total/Partial 正交, 可组合出四种类型。

[需要图示: 参见源文件 p.49–50 ISA 三角结构图、p.52 多重特殊化示例、p.55–57 Specialization/Generalization ER 图示例]

易错点: Disjoint 与 Overlapping 是关于“能否同时属于多个子类”, Total 与 Partial 是关于“是否必须属于某个子类”——这两个维度独立, 考试可能要求组合判断。

8 ER 设计方法论 (ER Design Methodology) [p.58–80]

8.1 三条设计原则 [p.59]

1. **相对原则**: 实体、属性、联系是对同一对象的不同抽象, 不同人/情境结果可能不同
2. **一致原则**: 同一对象在不同业务子系统中抽象结果必须一致
3. **简单原则**: 能归为属性处理的尽量归为属性

8.2 设计步骤 [p.64]

先局部 → 后整体 → 最后优化

8.2.1 局部 ER 设计 6 步 [p.66–68]

1. 确定局部应用范围
2. 确认实体集

3. 确认实体集间的联系集（穷举所有实体对）
4. 确认实体集的属性
5. 确认联系集的属性
6. 画出局部 E-R 图

8.2.2 全局集成：三类冲突 [p.71–72]

- **命名冲突**：同名异义（不同对象同名称）、同义异名（同一对象不同名称）
- **属性冲突**：值类型、取值范围、取值单位的冲突
- **结构冲突**：同一对象抽象层次不同（如工资在人事 = 属性，在财务 = 实体）；同一实体属性个数/次序不同

8.2.3 全局集成方法 [p.74]

两两集成法：以相同实体集为基准逐步合并，直到所有局部模型集成完毕

8.2.4 优化 [p.76–77]

三个条件：实体联系尽可能少、实体集所含属性尽可能少、实体集间联系无冗余

- 1:1 联系的两个实体集可合并
- 具有相同码的实体集可合并
- 冗余数据一般应消除，但高频查询时可保留 + 设完整性约束

8.3 完整实例 [p.65–77]

企业职工管理系统：人事管理 + 工资管理 + 项目管理 → 两两集成 → 全局优化（工资和职务均调整为实体集解决结构冲突）

/需要图示：参见源文件 p.68 人事管理局部 ER 图、p.69 工资管理 + 项目管理局部 ER 图、p.75 合并后全局初步 ER 图、p.61 工资属性变实体对比图、p.63 仓库属性变实体对比图/

解题技巧：遇到结构冲突（同一概念在不同局部中抽象层次不同），优先统一为实体集——这通常是最安全的做法，避免后续信息丢失。

8.4 学校练习 [p.78–80]

院 (School) ---1:N--- 班 (Class), 院 ---1:N--- 系 (Department), 系 ---1:N--- 教师 (Teacher), 教师 ---1:N--- 研究生 (GraduateStudent), 班 ---1:N--- 学生 (Student), 学生 ---M:N--- 课程 (Course) (属性：成绩/学期), 教师 ---M:N--- 课程 (属性：授课学期/地点)

8.5 在线学习平台完整设计实例 [p.41–43]

5 步设计法：识别核心实体 (User/Course/Instructor/Module/Quiz/Certificate) → 确定基本关系 (教授 1:N/包含 1:N/注册 M:N/获得 1:N) → 添加属性 → 设计关系细节 (Enrollment 关联实体/QuizSubmission 关联实体) → 特殊考虑 (Prerequisite 自反关系/UserRole 实体区分学生和管理员)

解题技巧：设计 ER 时先从最明显的核心实体入手，逐步扩展；遇到 M:N 且有附加属性的联系时创建关联实体。

9 UML 类图 (UML Class Diagrams) [p.81–85]

9.1 UML vs ER 图 [p.81–85]

- UML Class Diagram 对应 ER 图，但有多处差异
- 实体集：方框内含属性（非 ER 图中的单独椭圆）
- 二元联系：实体集间的连线，联系名写在连线旁
- 角色：写在连线旁靠近实体集的位置
- 非二元联系：使用菱形（与 ER 图相同）

9.2 UML 基数约束 [p.85]

- 格式：1..h, l= 最小, h= 最大参与数
- 1 等价于 1..1, * 等价于 0..*
- 关键差异：UML 约束位置与 ER 图恰好相反
- 易错点：UML 中 0..1 写在 E2 侧意味着“每个 E2 至多参与 1 个联系”，而在 ER 图中相应的箭头应指向 E1。切勿混淆方向

[需要图示：参见源文件 p.84 UML 与 ER 基数约束位置对比图、p.83 UML 类图示例]

10 ER 到关系模式的转换 (Reduction to Relation Schemas) [p.86–99]

10.1 数据库设计三阶段 [p.87]

概念设计 (Conceptual Design) → 逻辑设计 (Logical Design) → 物理设计 (Physical Design)

10.2 6 条核心转换原则 [p.96]

1. 实体集 → 一个关系模式
2. m:n 联系 → 一个关系模式 (PK 来自两个实体)
3. 1:n 联系 → 可与 n 端对应的关系模式合并 (将 1 端的关键字放入 n 端关系作为外键)

4. **1:1 联系** → 可与任意一端对应的关系模式合并（加入另一端的关键字）
5. **三个及以上实体间的多元联系** → 一个关系模式
6. **同一实体集间的自联系** → 按 1:1、1:n、m:n 三种情况分别处理

10.3 具体规则详解

- **强实体** → 关系模式保留所有简单属性 [p.88]
- **弱实体** → 关系模式包含标识强实体的主码 + 自身分辨符 + 自身属性。例: `payment(loan_number, payment_number, payment_date, payment_amount)` [p.88]
- **M:N 联系** → 独立关系模式，包含双方主码 + 联系描述属性。例: `borrower(customer_id, loan_number)` [p.89]
- **1:N 联系 (N 侧全部参与)** → 在 N 端加外键，指向 1 端主码 [p.91]
- **1:N 联系 (N 侧部分参与)** → 加外键可能导致大量 NULL 值 [p.92]
- **1:1 联系** → 任一侧加外键 [p.92]
- **弱实体标识联系的关系模式是冗余的**，不需要额外建立 [p.92]
- **复合属性** → 展开为多个简单属性列。例: `name` → `name.first_name, name.last_name` [p.93]
- **多值属性 M** → 独立关系模式 EM(实体主码, M), 每个值映射为单独一行。例: `employee_dependent_names(employee_id, dname)`。具体: 员工 123-45-6789 有家属 Jack 和 Jane → 映射为两行 (123-45-6789, Jack) 和 (123-45-6789, Jane) [p.93]

10.4 特殊化/概化转换 [p.94–95]

	Method 1	Method 2
做法	高层建一个模式 + 每个低层各建模式（含高层主键 + 本地属性）	每个实体集各自建模式（包含所有
优点	无冗余	访问低层只需查一个关系
缺点	访问低层需查两个关系	有重叠时属性冗余
适用	一般情况	total 特化时高层可不存，改用视图

10.5 综合练习 [p.97–99]

工厂 (工厂编号, 厂名, 地址) ↔ 产品 (产品编号, 产品名, 规格) m:n (属性: 计划数量 ≥ 100);
 工厂 ↔ 职工 (职工号, 姓名, 技术等级) 1:n (属性: 聘期, 工资)

转换得:

- 职工 (职工号, 姓名, 技术等级, 工厂编号, 聘期, 工资)
- 工厂 (工厂编号, 厂名, 地址)
- 产品 (产品编号, 产品名, 规格)
- 生产 (工厂编号, 产品号, 数量)

SQL 建表含:PRIMARY KEY, FOREIGN KEY ON DELETE CASCADE/RESTRICT, CHECK(数量 ≥ 100) [p.99]

11 弱实体复习要点 [p.106]

- 若实体 x 的存在依赖于实体 y 的存在, 则 x 是从属实体 (Subordinate Entity), y 是主导实体 (Dominant Entity)
- 删除 dominant 实体时, 必须同时删除其关联的所有 subordinate 实体 (级联删除)